

Project Report: Model-View-Controller (MVC) Architecture Implementation

1. MVC Implementation in User & Authentication Module

In our project, we used the MVC pattern to keep our code organized. I will explain how MVC works in the **User Login** feature, which is part of the User & Authentication Module I worked on.

The Model (Data)

The Model represents our database structure. In this project, we used Mongoose for MongoDB. The user model is in `backend/models/User.js`.

- **Role:** It checks if the entered email is valid and securely hashes the password before saving it. It also has a function (`comparePassword`) to check if the password matches during login.

The View (UI)

The View is what the user sees. We used React for the frontend interface. The login view is located at `frontend/src/components/member1/Login.jsx`.

- **Role:** `Login.jsx` shows a form where users type their email and password. When the user clicks login, the View sends this data to the backend. If successful, it saves the login token and redirects the user to their dashboard.

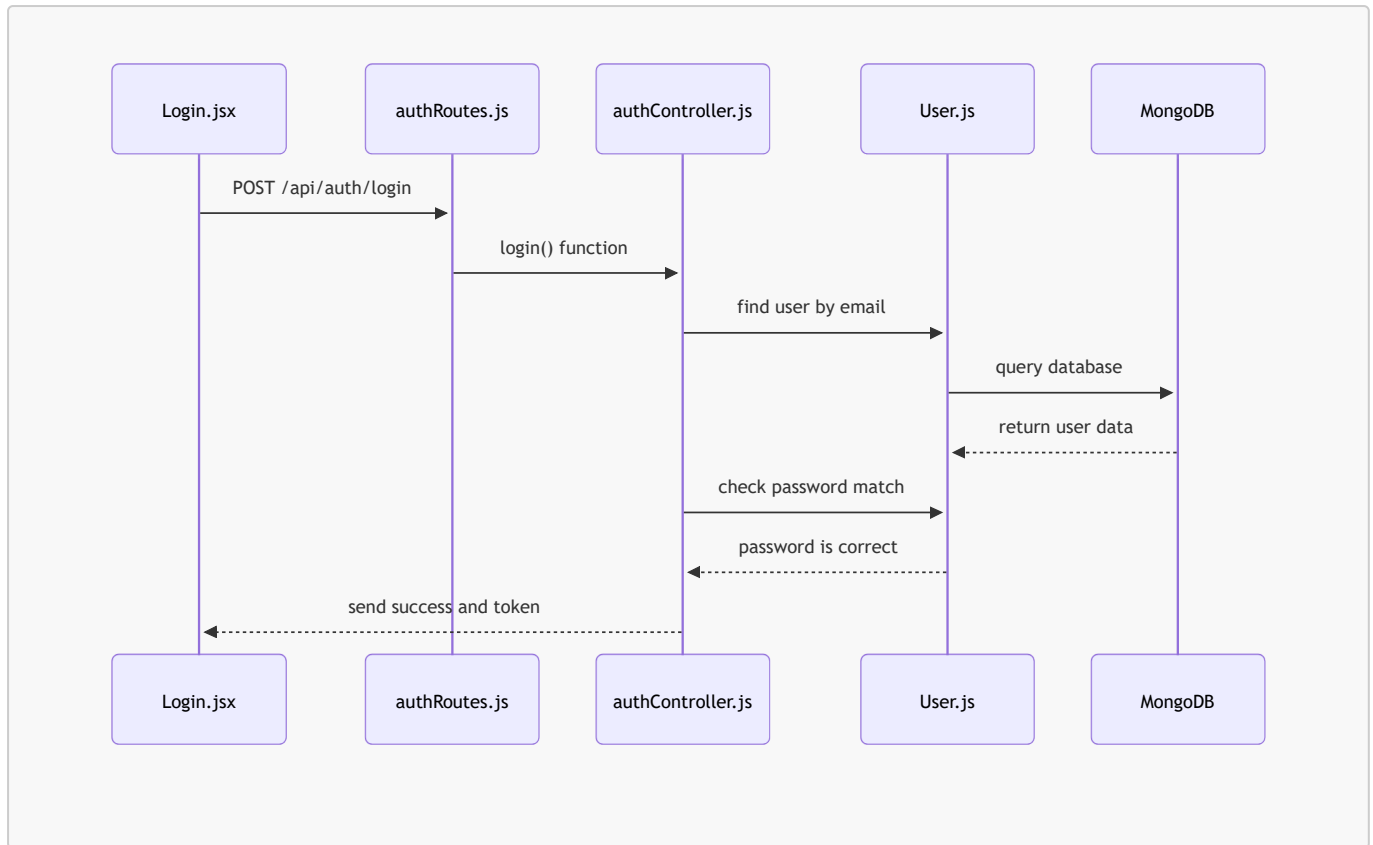
The Controller (Logic)

The Controller connects the View and the Model. The controller logic is in `backend/controllers/authController.js`, and the routes are in `backend/routes/authRoutes.js`.

- **Role:** The router passes the login request to `authController.js`. The controller then asks the `User.js` Model if the email exists and if the password is correct. If everything is good, the controller sends a success message and a token back to the View. If not, it sends an error message.

2. MVC Flow Diagram

Here is a simple flow diagram showing what happens when a user logs in.



Flow Explanation

1. The user enters their email and password in **Login.jsx** and hits submit.
2. The request hits **authRoutes.js**, which calls the `login` function in the controller.
3. The **authController.js** takes the data and asks the **User.js** model to find the user in the database.
4. The controller then uses the model to check if the passwords match.
5. Finally, the controller sends the result (success or error) back to **Login.jsx**, which updates the screen.

This setup keeps our React UI completely separate from our database logic.

3. Application Screenshots (Views)

Here are the screenshots of our frontend Views:

Landing Page:



Login View (Login.jsx):

